

Python Static Security Analyzer

Test Case Documentation

Test Case: `vulnerable_test.py`

This file is a **test case for the main Python Static Security Analyzer project**. It is intentionally written with several insecure and unused-code patterns so that the analyzer can be tested against known examples.

The test case is not intended to be used as a real application. Its purpose is to provide input to the security scanner and verify that the scanner correctly identifies the security issues and code-quality issues it was designed to detect.

Usage

Save the following code as **`vulnerable_test.py`** and run the main scanner against it:

```
python scanner.py vulnerable_test.py
```

The scanner analyzes the file statically using Python's Abstract Syntax Tree (AST) and reports the detected findings in the terminal. It also saves the results to **`report.txt`**.

Purpose of the Test Case

The code contains examples corresponding to the detection rules implemented in the main security analyzer:

- **Hard-coded password:** `password = "admin123"` is included to test detection of a hard-coded credential.
- **`eval()`:** Tests detection of a potentially dangerous dynamic execution function.
- **`exec()`:** Tests detection of dynamic Python code execution.
- **`os.system()`:** Tests detection of a system command execution call.
- **`subprocess.call()`:** Tests detection of a subprocess command execution call.
- **Unused variables:** Variables such as `d`, `x`, and `y` are included to test the analyzer's unused-variable detection.

Expected Findings

When this test case is scanned, the main analyzer is expected to identify the following categories of findings:

HIGH — Dangerous calls: `eval()`, `exec()`, `os.system()`, and `subprocess.call()`.

MEDIUM — Hard-coded secret: the `password` variable.

LOW — Unused variables: `d`, `x`, and `y`.

Complete Test Case Code

```
import os
```

```
import subprocess

# hardcoded password
password = "admin123"

# variables
a = 10
b = 20
c = 30
d = 40

print(a)

# dangerous functions
eval("a + b")
exec("print(c)")

os.system("echo Hello")
subprocess.call(["echo", "Hello"])

# used variable
result = a + b
print(result)

# unused variables
x = 100
y = 200
```